

B

C

Bachelorarbeit

Compileranbindung für die
Programmier-Lern-Plattform
CodeTask

S

Bachelorarbeit

Compileranbindung für die Programmier-Lern-Plattform CodeTask

von

Cilas Handloser

zur Erlangung des akademischen Grades

Bachelor of Science

in Angewandter Informatik

an der Hochschule Konstanz Technik, Wirtschaft und Gestaltung

Matrikelnummer: 307610

Adresse: Im Türmle 11, 78224 Singen

Ausstellungsdatum: 01. April 2026

Abgabedatum: 30. Juni 2026

Erstbetreuer: **Prof. Dr. Marko Boger**

Zweitbetreuer: **Prof. Dr. Pascal Laube**

Eine elektronische Version dieser Bachelorarbeit ist verfügbar unter <https://github.com/Ciilas/htwg-latex-BA-Cilas>.

Abstract

Diese Bachelorarbeit beschäftigt sich mit der Weiterentwicklung der Check-API und dem damit verbundenen System der Programmierlernplattform CodeTask. CodeTask ist eine an der HTWG Konstanz entwickelte Website, zum Bearbeiten, Lösen und Lernen von Programmieraufgaben in der Programmiersprache Scala. CodeTask ermöglicht dabei direkte Ausführung und Fehlerbehandlung von Code. Für diese Aufgabe kommt der isolierte Service Check-API zum Einsatz, welcher darauf ausgelegt ist, Code in einem aktiven System ausführen zu können. Zu Beginn der Arbeit verfügte die Check-API bereits über die Fähigkeit, Code zusammen mit öffentlichen Tests ausführen und evaluieren zu können. Die Check-API verfügte zu diesem Zeitpunkt allerdings nur über eine eingeschränkte Rückmeldung, besaß keine Unterstützung für versteckte Tests, war nur eingeschränkt gegen problematische oder bösartige Einreichungen abgesichert und konnte durch eine einfache, unendliche Schleife zum Stillstand kommen.

Ziel der Arbeit war es, die Check-API robuster, aussagekräftiger und sicherer zu machen. Dafür wurde das System um die Möglichkeit erweitert, versteckte Tests zu nutzen, sodass gezielte Umgehungen der öffentlichen Tests erschwert werden. Zusätzlich wurde die Check-API mit einer Sicherheitsprüfung von Code auf unsichere Befehle ausgestattet, um bekannte, kritische Befehle vor der Ausführung zu erkennen und ausgewählte Angriffs- bzw. Fehlerfälle zu verhindern. Zudem wurden Fehlermeldungen systematisch ausgewertet und aufbereitet, um eine verbesserte Fehlersuche für den Nutzer zu ermöglichen.

Die Evaluation zeigt, dass in den getesteten Fällen keine Inhalte der versteckten Tests an Nutzer zurückgegeben wurden. Des Weiteren werden Kompilierungsfehler, fehlgeschlagene öffentliche Tests, fehlgeschlagene versteckte Tests, Zeitüberschreitungen und Sicherheitsverstöße korrekt erkannt, verarbeitet und strukturiert an den Nutzer weitergegeben. Die Performance-Messungen legen eine klare Verbesserung der Leistung offen, zeigen jedoch deutlich, dass die Ausführung von Nutzercode rechenintensiv bleibt und für hohe Last geeignete Skalierungsmaßnahmen erforderlich sind.

Inhaltsverzeichnis

Abbildungsverzeichnis	ix
Tabellenverzeichnis	xi
Listingsverzeichnis	xiii
Abkürzungsverzeichnis	xv
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	2
1.3 Ziel der Arbeit	2
1.4 Aufbau der Arbeit	3
2 Grundlagen und Technologien	5
2.1 Online-Judge-Systeme	5
2.1.1 Architektur eines Online-Judges	6
2.1.2 Der Prozessablauf	7
2.1.3 Vergleich bestehender Systeme	8
2.2 Containerisierung mit Docker	10
2.2.1 Das Docker-Prinzip (Images und Container)	10
2.2.2 Isolation und Sicherheit (Sandboxing)	11
2.3 Orchestrierung mit Kubernetes	12
2.3.1 Grundlagen der Container Verwaltung	12
2.3.2 Hochverfügbarkeit und Self-Healing	12
3 Anforderungen	15
3.1 Analyse des Altsystems und Problemstellung	15
3.2 Zielsetzung und Systemgrenzen	17
3.3 Funktionale Anforderungen	17
3.4 Nicht-funktionale Anforderungen	18
4 Konzept und Architektur	19
4.1 Überblick über die Zielarchitektur	19
4.2 Verarbeitungsablauf einer Einreichung	20

4.3	Konzept zur Integration versteckter Tests	20
4.4	Konzept zur isolierten Codeausführung	21
4.5	Konzept zur Ergebnis- und Fehlerbehandlung	22
5	Implementierung	25
5.1	Ausgangszustand der Check-API und Aufgabenverarbeitung	25
5.1.1	Übungsaufgabenverwaltung	25
5.2	Erweiterung des Parsers und der Testdatenverarbeitung	26
5.3	Bereitstellung und Zuordnung von versteckten Tests.	28
5.4	Anpassung der Check-API	28
5.5	Zeitüberschreitung.	30
5.6	Fehlerbehandlung und Strukturierte Ergebnissrückgabe	30
6	Evaluation	33
6.1	Ziel der Evaluation.	33
6.2	Evaluationsumgebung	33
6.3	Funktionale Evaluation	34
6.4	Evaluation der Robustheit und Verfügbarkeit	34
6.5	Performance-Evaluation	35
6.6	Bewertung der Ergebnisse	38
6.7	Grenzen der Evaluation.	39
7	Fazit	41
7.1	Zusammenfassung	41
7.2	Bewertung der Ergebnisse	42
7.3	Ausblick	42
	Literatur	45

Ehrenwörtliche Erklärung

Hiermit erkläre ich, Cilas Handloser, geboren am 22. Oktober 2002 in 78224 Singen,

1. dass ich meine Bachelorarbeit mit dem Titel:
„Compileranbindung für die Programmier-Lern-Plattform CodeTask“
in der Fakultät Informatik der HTWG unter Anleitung von Professor Doctor Marko Boger selbständig und ohne fremde Hilfe angefertigt habe und keine anderen als die angeführten Hilfen benutzt habe;
2. dass ich die Übernahme wörtlicher Zitate, von Tabellen, Zeichnungen, Bildern und Programmen aus der Literatur oder anderen Quellen (Internet) sowie die Verwendung der Gedanken anderer Autoren an den entsprechenden Stellen innerhalb der Arbeit gekennzeichnet habe.
3. dass ich bei der Erstellung der Arbeit KI-basierte Werkzeuge ausschließlich unterstützend bei der Programmierung, Ideenfindung sowie der sprachlichen Überarbeitung (Formulierung, Grammatik und Rechtschreibung) eingesetzt habe. Die sprachlichen Überarbeitungen der KI wurden von mir überprüft. Die inhaltliche Ausarbeitung der Arbeit erfolgte eigenständig. Sämtliche Aussagen, Ergebnisse und Schlussfolgerungen wurden von mir selbst erstellt, geprüft und verantwortet.
4. dass von mir selbst eingereichte, nicht vom Druckservice erstellte Exemplare in Papierform mit der hochgeladenen PDF-Version übereinstimmen.

Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Konstanz, 30. Juni 2026

Abbildungsverzeichnis

4.1	Konzept-Architektur	20
6.1	Performance Vergleich zwischen der lokalen, Docker-Compose und der Kubernetes Umgebung mit jeweils 100 gleichzeitigen Nutzern, im Vergleich die 99. Perzentil der Antwortzeit	36
6.2	Performance Vergleich zwischen der lokalen, Docker-Compose und der Kubernetes Umgebung mit jeweils 100 gleichzeitigen Nutzern, im Vergleich die 95. Perzentil der Antwortzeit	37
6.3	Performance Vergleich zwischen der lokalen, Docker-Compose und der Kubernetes Umgebung mit jeweils 100 gleichzeitigen Nutzern, im Vergleich die 50. Perzentil der Antwortzeit	37
6.4	Performance Vergleich zwischen der Docker-Compose und der Kubernetes Umgebung mit jeweils 50 und 100 gleichzeitigen Nutzern	38

Tabellenverzeichnis

2.1	Vergleich der Online-Judge-Plattformen	10
5.1	Fehlerbehandlung und Rückgabemodell der Check-API	32
6.1	Funktionale Testfälle der Check-API	35

Listingsverzeichnis

2.1	Beispiel eines Dockerfiles für den Kompiler-Service	11
3.1	Beispiel-Code einer CodeTask Übungsaufgabe, ohne Nutzercode	16
5.1	Koan Ausschnitt ohne versteckte Tests	27
5.2	Das Feld "code" nach dem Bestehenden Parser	27
5.3	Koan Ausschnitt mit versteckten Tests	28

Abkürzungsverzeichnis

API Application Programming Interface.

CPU Central Processing Unit.

HTWG Hochschule Konstanz Technik, Wirtschaft und Gestaltung.

JDK Java Development Kit.

JSON JavaScript Object Notation.

JVM Java Virtual Machine.

K8s Kubernetes.

LDAP Lightweight Directory Access Protocol.

SPOJ Sphere Online Judge.

1

Einleitung

1.1. Motivation

Um Programmieren richtig lernen zu können, muss die Theorie auch in der Praxis angewendet werden. Das wird aber besonders am Anfang durch aufwendiges Herunterladen von Codeeditoren, Einrichten der Programmiersprache und dem System erschwert und schreckt ab. Dafür wurde an der HTWG Konstanz die Lernplattform CodeTask entwickelt, auf der Studenten strukturiert die Konzepte rund ums Programmieren lernen können. CodeTask setzt dabei auf einfach aufgebaute Aufgaben. Der Großteil der Aufgaben fängt mit einem kurzen Video über die aktuelle Aufgabe an, daraufhin folgt ein kleiner Infotext, in welchem die Aufgabe noch mal erklärt wird. Nach dem Informationstext wird der Student bzw. der Nutzer vor unterschiedliche Aufgabentypen gestellt, die er lösen kann. Darunter auch der CodeTask Aufgabentyp, bei welchem der Student eigenständig in einem Codeeditor auf der Website Code programmieren und validieren lassen kann. Diese Lernplattform sorgt dafür, dass neue Studenten oder Personen, die erste Erfahrungen im Programmieren sammeln, nicht sofort mit überflüssigen Informationen überschüttet werden und sich erst mal auf die wirklich essenziellen Bereiche der Programmierung fokussieren können. Damit solch ein System funktionieren kann, muss das System in der Lage sein, den Code, welcher von dem Nutzer geschrieben wird, ausführen und gegen vordefinierte Tests prüfen zu können und dem Nutzer ein klares Ergebnis zu liefern, mit dem der Nutzer genau sieht, ob er alles richtig gemacht hat, oder, wenn es ein Problem gab, warum es zu diesem Problem gekommen ist. CodeTask kann genau das, und zwar durch die Check-API. Ein Service, welcher den Nutzercode zusammen mit den für diese Aufgabe vorgesehen Tests übergeben bekommt,

diesen überprüft, ausführt und zum Schluss ein Ergebnis zurück gibt, welches für den Nutzer möglichst verständlich ist.

1.2. Problemstellung

Solch ein System bringt aber einige kritische Probleme mit sich, welche richtig gehandhabt werden müssen, um unerwünschtes Verhalten, wie Systemausfälle oder lange Wartezeiten, zu verhindern. Zu Beginn dieser Arbeit war die Check-API bereits ein eigener Service und behandelte schon einige dieser kritischen Probleme, allerdings nicht alle und manche auch nicht ausreichend genug. Durch die Entwicklung der Check-API als eigener Service wurde so schon vor dieser Bachelorarbeit das größte Problem sehr stark eingeschränkt. Das Ausführen von Code während der Laufzeit ist für solch ein System eines der größten Sicherheitsrisiken, da willkürlicher Code mit vollständiger Berechtigung auf das System ausgeführt werden kann. Durch die Isolation in einen eigenen Service wird dieses Problem schon stark eingeschränkt, da die Auswirkungen problematischer Codeausführung auf die Check-API begrenzt werden und so die umliegenden Systeme nicht direkt betroffen sind. Aber nicht nur das Ausführen von Code ist in so einem System ein Problem. Besonders auf den Lernprozess der Nutzer ist es relevant, diese richtig testen zu können. Bisher unterstützte das System lediglich öffentliche Testfälle, welche größtenteils sehr leicht zu umgehen sind und so die eigentliche Lösung nicht wie erwünscht gelöst wird. Des Weiteren bekam der Nutzer nur mitgeteilt, ob die Aufgabe korrekt gelöst wurde oder einen Fehler hatte, dabei wurde aber nicht klar übermittelt, was für ein Fehler es ist und warum er möglicherweise aufgetreten ist. Als letztes Problem kann die Ausführung von Nutzercode ein Risiko für die Stabilität des Systems darstellen. Programme können ineffizient programmiert oder, schlimmer noch, Endlosschleifen beinhalten. Ohne eine Kontrolle über die Laufzeit kann die Ausführung von solch einem Programm extrem viele Ressourcen verbrauchen oder sogar das gesamte System extrem ausbremsen oder sogar unbrauchbar machen. Problematisch wird so ein Verhalten vor allem, wenn die Plattform von mehreren Nutzern gleichzeitig genutzt wird.

1.3. Ziel der Arbeit

Das Ziel dieser Bachelorarbeit ist die Weiterentwicklung der bestehenden Check-API. Dabei sollen die eben genannten Probleme verbessert oder vollständig gelöst werden.

Die Architektur soll dabei nicht ersetzt, sondern gezielt erweitert werden, sodass eine Einreichung von Nutzercode sicher, robust und aussagekräftig verarbeitet werden kann, um eine möglichst effektive und angenehme Lernumgebung zu schaffen.

Ein Schwerpunkt liegt auf der Unterstützung von versteckten Tests. Diese ermöglichen das Überprüfen des Nutzercodes, ohne dass der Nutzer weiß, wie dieser Test aussieht. Dadurch wird erschwert, dass der Nutzer seine Lösung gezielt an die sichtbaren Tests anpasst. Dabei muss beachtet werden, dass der Nutzer zu keinem Zeitpunkt diese versteckten Tests einsehen kann, um auch hier eine gezielte Umgehung zu verhindern. Des Weiteren ist eine verbesserte Fehlerbehandlung und Ausgabe von Relevanz. Sollte ein Nutzer nur mitgeteilt bekommen, dass ein Fehler vorliegt, aber nicht gesagt wird, warum dieser Fehler aufgetreten ist, verliert der Nutzer die Lust, weil er ohne Weiteres eventuell nicht weiterkommt. Daher ist es wichtig, die möglichen Fehlerquellen zu erkennen und diese auf eine individuell angepasste Weise zu handhaben, um dem Nutzer möglichst klar mitzuteilen, was das Problem ist, ohne dem Nutzer die genaue Lösung vorzulegen. Darüber hinaus muss die Robustheit weiter verbessert werden. Nicht terminierende oder problematische Einreichungen dürfen den Service nicht permanent außer Betrieb nehmen oder blockieren. Dafür muss vor der Ausführung geprüft werden, ob es in dem eingereichten Code kritische Befehle gibt, welche dem Service schaden können, und zusätzlich muss die Dauer der Ausführung überwacht werden. Weiterhin wird die Leistung des Systems untersucht, um festzustellen, wie das System schneller und leistungsfähiger gemacht werden kann.

1.4. Aufbau der Arbeit

Kapitel 2 beschreibt die Grundlagen und Technologien, die für das Verständnis der Arbeit notwendig sind. Dazu wird zusätzlich auf vergleichbare Systeme eingegangen, um einen Vergleich mit der Lösung von CodeTask zu schaffen.

In Kapitel 3 wird auf das bestehende System sowie die Problemstellung eingegangen. Des Weiteren werden die Anforderungen dieser Arbeit beschrieben. Darunter die funktionalen und nicht funktionalen Anforderungen, die sich für die Check-API und das umliegende System ergeben.

Kapitel 4 geht auf das Konzept und die Zielarchitektur der erweiterten Check-API ein. Es geht darauf ein, wie die zuvor definierten Anforderungen umgesetzt werden können und welche Rolle die Check-API in dem System übernimmt.

Die genaue Umsetzung wird in Kapitel 5 erläutert. Dort wird die Anpassung des Parsers zur Unterstützung von versteckten Tests, die Zusammensetzung einer Anfrage

an die Check-API, die Umsetzung der Geheimhaltung der versteckten Tests sowie die verbesserte Fehlerbehandlung und Robustheit beschrieben.

Kapitel 6 evaluiert die umgesetzte Lösung. Dafür wird die Fehlerbehandlung aufgeschlüsselt und bewertet, sowie die Check-API in unterschiedlichen Umgebungen auf Performance untersucht und verglichen.

Abschließend fasst Kapitel 7 die Ergebnisse dieser Bachelorarbeit zusammen. Zusätzlich werden die Ergebnisse bewertet und ein Ausblick auf mögliche Verbesserungen und Weiterentwicklungen geschaffen.

2

Grundlagen und Technologien

In diesem Kapitel werden die Grundlagen und die verwendeten Technologien erläutert, die für die Konzeption und Umsetzung der erweiterten Check-API für CodeTask von Bedeutung sind. Zunächst wird das Konzept von Online-Judge-Systemen im Allgemeinen betrachtet, bevor auf die spezifischen Isolationsmechanismen eingegangen wird.

2.1. Online-Judge-Systeme

Ein Online-Judge-System ist eine Plattform, welche Programmieraufgaben zur Verfügung stellt, die von Nutzern gelöst werden können. Dazu wird der Code gegen vordefinierte Testfälle geprüft. Durch solch ein System können sich Studenten, Schüler, aber auch Berufstätige in unterschiedlichen Programmiersprachen zu allen möglichen Themen fortbilden, ohne sämtliche Systeme selbst aufsetzen zu müssen. Zudem sorgt solch ein System für eine einheitliche, objektive Bewertung – gleiche Eingabe, gleiche Ausgabe, es gibt keine Sympathie, die bevorzugen oder benachteiligen kann. Zusätzlich bekommt man ein sofortiges Ergebnis. Ein Computer kann die Aufgaben schneller bewerten als jeder Mensch. Ein Online-Judge kann in unterschiedlichsten Möglichkeiten genutzt werden. Es kann regulär zum Lernen/Fortbilden genutzt werden, in Wettbewerben, in denen man gemessen werden kann, oder als Portfolio zum Offenlegen aktueller Fähigkeiten.

2.1.1. Architektur eines Online-Judges

Ein jedes Online-Judge-System besteht aus einer Handvoll Bausteinen, die für einen reibungslosen Betrieb vonnöten sind.

User-Interface

In irgendeiner Form muss der Nutzer seinen Code abgeben können. Hier kommt das Frontend, in der Regel als eine Website, ins Spiel. In einem Textfeld wird der Code eingegeben und zum Prüfen an das Backend geschickt.

Aufgaben-Management

Damit das System weiß, welche Aufgaben und Nutzer es gibt, wird eine Datenbank benötigt. In dieser werden sämtliche notwendigen Informationen permanent abgespeichert. Darunter fallen auch Ergebnisse von Nutzern oder Statistiken. Damit die Aufgaben im Frontend angezeigt werden können, müssen die Informationen sowohl erstmals in der Datenbank gespeichert als auch kontinuierlich gelesen werden. Dafür sorgt ein Management Service. Dieser Service verbindet das Frontend mit der Datenbank und ermöglicht die notwendige Kommunikation.

Judging-Engine / Compiler

Jedes dieser Systeme muss den Programmcode des Nutzers ausführen, um zu verifizieren, dass dieser das korrekte Ergebnis liefert. Das passiert mit einer Judging-Engine. Diese nimmt die Anfragen mit dem Code entgegen und führt den Code aus. Das Ganze passiert dabei in der Regel asynchron, damit das System auch bei vielen Anfragen nicht zusammenbricht. Sobald die Judging-Engine den Code ausgeführt hat und ein Ergebnis erstellt hat, wird dieses zurück geschickt.

Sandbox

Der kritischste Teil eines solchen Systems liegt in der sogenannten dynamischen Code-Ausführung. In diesem Kontext bedeutet dynamische Code-Ausführung, dass ein System zur Laufzeit von Nutzern eingereichten Code entgegennimmt, kompiliert und ausführt.

Die dynamische Code-Ausführung ist dabei sehr häufig ein großes Risiko, da der Code vollen Zugriff auf das System hat, in dem er läuft. Das kann bedeuten, dass der Code gefährliche Befehle, wie zum Beispiel `sys.exit()`, ausführen kann und somit das gesamte Programm beenden oder sogar Dateien auslesen kann. Um diesem Problem zu entgehen, kann man so ein System in einer Sandbox aufbauen. Dabei ist eine Sandbox ein abgesichertes System, das nur eingeschränkt arbeiten kann. Eine Sandbox kann die Auswirkungen problematischer Codeausführungen begrenzen. Es ist allerdings keine Garantie für ein vollständig sicheres System und benötigt dennoch eine vollständige Sicherheitsanalyse.

2.1.2. Der Prozessablauf

Einreichung

Die Reise startet im Frontend. Der Nutzer wählt eine Aufgabe aus, die er lösen möchte. Daraufhin wird diese Aufgabe mit Tests und eventuell vorgegebenem Code aus der Datenbank geladen und für den Nutzer angezeigt. Sobald der Nutzer fertig programmiert hat, kann er seine Lösung abschicken. Durch das Abschicken werden alle relevanten Daten an das Backend geschickt. Im Backend werden weitere benötigte Daten aus der Datenbank geladen und zusammen an die Judging-Engine gegeben.

Validierung

Ist der Code überhaupt vollständig beziehungsweise syntaktisch korrekt? Das wird je nach Programmiersprache unterschiedlich entschieden. In Sprachen wie C, Java oder Scala ist hier ein vorläufiges Überprüfen des Codes hilfreich. Durch Untersuchen des Codes auf bestimmte Schlüsselwörter kann so frühzeitig erkannt werden, ob der Code bössartig ist.

Kompilierung

Ist die Validierung erfolgreich, kann der Code kompiliert werden. Dafür kann ein vorhandener Kompiler genutzt oder eigener geschrieben werden. Damit der Kompiler den Code übersetzen kann, müssen von dem Kompiler noch einige Prüfungen stattfinden. Darunter prüfen, ob die Vokabeln korrekt sind, also ob die Schlüsselwörter wie `val` oder `if` richtig geschrieben wurden. Daraufhin wird die Syntax geprüft, also ob der Satzbau korrekt ist, zum Beispiel ob eine Klammer oder ein Semikolon vergessen wurde [Ses17]. Und zum Schluss findet die semantische Prüfung statt, hier wird überprüft, ob der Code Sinn ergibt. Einen Text durch eine Zahl zu teilen, wäre dabei ein Szenario, bei dem diese Prüfung anschlagen würde. Ist bis hierhin alles fehlerfrei, übersetzt der Scala-Kompiler den Code in JVM-Bytecode, welcher anschließend auf der Java Virtual Machine ausgeführt werden kann [EPF25].

Ausführung

Der Code ist übersetzt und kann ausgeführt werden. Der Code liefert am Ende ein beliebiges Ergebnis. Dieses muss jedoch überprüft werden, damit beurteilt werden kann, ob der Code auch die gestellte Aufgabe löst.

Bewertung

Um den Code bewerten zu können, brauchen wir Testfälle, die mit dem Code zusammen kompiliert wurden. Diese vergleichen das erzeugte Ergebnis des Codes mit der erwarteten Lösung.

Rückmeldung

Stimmt das Ergebnis mit dem Test überein, so wird dem Nutzer dies mitgeteilt. Ist das Ergebnis falsch oder der Code zu komplex, sodass das Programm zu lange ausgeführt werden muss, wird auch das dem Nutzer beigebracht.

2.1.3. Vergleich bestehender Systeme

HackerRank und LeetCode

LeetCode [Lee] und HackerRank [Hac], sind aktuell die größten Online-Judge-Systeme. Beide Unternehmen bieten auf ihren Websites mehrere hundert unterschiedliche Programmieraufgaben in unterschiedlichen Programmiersprachen an. Ihr Ziel ist es, Studenten, Auszubildende, Quereinsteiger, aber auch angestellte Entwickler weiterzubilden. Die Aufgaben von LeetCode sind dabei sehr stark an die gestellten Aufgaben von riesigen Tech Firmen wie Google, Meta, Amazon und Co. angelehnt. Damit ist LeetCode ideal geeignet, um sich auf Codeinterviews vorzubereiten. LeetCode fokussiert sich dabei stark auf algorithmische Aufgaben, wie Data Structures oder Algorithms. HackerRank auf der anderen Seite wird zwar genau so für Interviews direkt genutzt, bietet aber häufiger Coding Challenges und Wettbewerbe an, in denen sich die Nutzer untereinander vergleichen, messen und motivieren können. Zudem ist das Angebot an Themen auch breiter. Datenbanken, KI, Sicherheit und Linux und weitere Aufgabenbereiche werden hier angeboten. Beide Unternehmen müssen mit einer hohen Anzahl gleichzeitiger Nutzer und Anfragen umgehen können. Für Online-Judge-Systeme sind parallele Verarbeitung, Warteschlangen und isolierte Ausführungsumgebungen relevant [Was+18]. Diese Mechanismen ermöglichen es, eingereichten Code getrennt vom restlichen System auszuführen und Lastspitzen besser verarbeiten zu können. Im Vergleich ist CodeTask wesentlich kleiner skaliert und auf den Einsatz innerhalb von Hochschulen gedacht. Dennoch entstehen ähnliche technische Anforderungen, da auch hier Nutzercode zuverlässig, isoliert und mit verständlicher Rückmeldung ausgeführt werden muss.

Sphere Online Judge (SPOJ)

SPOJ [Spo], aus den frühen 2000er Jahren, ist eines der ersten Judging Systeme. SPOJ hat es jedem ermöglicht, eigene Aufgaben zu erstellen und online hochzuladen. So konnte Code in über 40 Programmiersprachen, darunter auch Exoten wie Intercal oder Brainfuck, getestet werden. Allerdings musste für jede Sprache ein Compiler oder Interpreter eingebaut sein, was diese extreme Vielfalt extrem wartungsintensiv macht. SPOJ hat damit aber den Grundstein für das Grundprinzip Code → Test → Ergebnis gelegt, welches 20 Jahre später noch genau so angewandt wird. Im Vergleich zu LeetCode liegt bei SPOJ der Fokus auf reiner Informatik. Hier geht es darum, arithmetische

Probleme zu lösen, und weniger darum, sich für das nächste Code-Interview vorzubereiten. SPOJ setzt auf ein sehr striktes Bewertungssystem, das auch auf Faktoren wie Zeit- und Speicherlimits achtet, um die Effizienz des Codes zu prüfen.

CodeTask

In unserem hochschuleigenen Projekt CodeTask, um das es in dieser Arbeit geht, liegt der Fokus vorerst auf der Lehre der Studenten in der Programmiersprache Scala. CodeTask soll sowohl in Vorlesungen als auch in Prüfungen von den Dozenten und Studenten genutzt werden. Dafür können Lehraufgaben direkt vom Dozenten oder Tutoren selbst erstellt und bereitgestellt werden. Das Ziel ist nicht, dass der Student ein Wahr oder Falsch bekommt, sondern eine klare Antwort, die für einen idealen Lernfortschritt sorgt. Aktuell ist CodeTask noch in der Entwicklung und primär auf die Nutzung innerhalb des Hochschulnetzwerks der HTWG-Konstanz begrenzt. Damit Datenschutz und, viel wichtiger, die Kontrolle über studentische Daten gewährleistet werden können, läuft die Applikation aktuell auf Hochschul eigenen Servern und nicht, wie bei LeetCode, auf riesigen externen Cloud-Infrastrukturen. Ein Nachteil liegt dabei darin, dass das System effizient auf Hochschul-Servern funktionieren muss. Dafür ist es allerdings einfacher, auf Services wie LDAP (Hochschul-Login) und weitere Tools der Hochschule zuzugreifen. CodeTask ist eine Eigenentwicklung der HTWG-Konstanz, welche zum aktuellen Stand vorwiegend auf Scala-Übungen ausgelegt ist. Die im Rahmen meiner Bachelorarbeit weiterentwickelte Check-API bildet dabei die entscheidende technische Grundlage, um die Programmiersprache Scala in das System zu integrieren und gleichzeitig eine universelle Schnittstelle für die zukünftige Einbindung weiterer Programmiersprachen zu ermöglichen.

Vergleichstabelle und Analyse

Zur Verdeutlichung der Unterschiede zwischen den vorgestellten Systemen und der Einordnung von CodeTask dient die folgende Tabelle 2.1. Diese stellt die kommerziellen Plattformen, historische Pioniere und die spezialisierte Hochschullösung anhand ausgewählter Kriterien gegenüber.

Wie in Tabelle 2.1 zu erkennen ist, belegt CodeTask eine etwas andere Rolle. Während HackerRank und LeetCode stark auf Skalierbarkeit für Millionen von Nutzern ausgelegt sind, liegt der Fokus bei CodeTask auf der direkten Einbindung des Lehrplans und der Kontrolle über die studentischen Daten. Die in dieser Arbeit weiterentwickelte Anbindung nutzt dabei mit Docker und Kubernetes Clustern eine Technologie, die verbreitet zur Containerisierung und Orchestrierung von Diensten eingesetzt wird.

Tabelle 2.1: Vergleich der Online-Judge-Plattformen

Kriterium	HackerRank / LeetCode	SPOJ	Pro-	CodeTask
Zielgruppe	Globales Recruiting / Wettbewerbe	Competitive programming	Pro-	Hochschullehre (HT-WG)
Infrastruktur	Cloud-basiert (global)	Server-basiert (zentral)		Lokal (Hochschul-Server)
Sprachvielfalt	Sehr hoch (> 30)	Extrem hoch (> 40)		Fokus auf Lehre (Scala)
Isolation	Cloud-Container	Klassische Jails		Docker-Container / Kubernetes Cluster
Open Source	Nein (Kommerziell)	Nein		Ja (Eigenentwicklung)

2.2. Containerisierung mit Docker

2.2.1. Das Docker-Prinzip (Images und Container)

Um die Ausführung von Nutzercode für das System sicher zu machen, wird Isolation benötigt. Es braucht eine Möglichkeit, Code auf einem Server ausführen zu können, ohne dabei das gesamte System durch fehlerhaften oder böartigen Code zu gefährden. Hier kommt in unserem Fall Docker [Doc26] zum Einsatz.

Docker-Images

Docker-Images sind der Grundstein von Docker. Sie können mithilfe einer sogenannten Dockerfile erstellt werden. Listing 2.1 zeigt hier solch eine Dockerfile. Als Erstes wird ein passendes Image ausgewählt. Ein Image ist in diesem Fall ein eigenes kleines Betriebssystem, welches später in den erstellbaren Containern ausgeführt wird und die Basis für Codeausführung oder andere Aufgaben bildet. Nach dem das Image ausgewählt ist, wird ein Ordner erstellt, der das Quellverzeichnis darstellt. Daraufhin werden alle notwendigen Dateien in dieses Quellverzeichnis kopiert und zum Schluss kann ein Befehl ausgeführt werden, welcher z. B. den Code aus dem Quellverzeichnis kompiliert und ausführt. In einem Docker-Image befinden sich dann all diese Informationen. Die Dockerfile aus Listing 2.1 stellt dabei eine leicht veränderte Version der Dockerfile

dar, mit welcher das Image für die Entwicklungsumgebung der Check-API von Code-Task erstellt wird. Das Produktions-Image ist dabei etwas komplexer und optimiert für eine effizientere Nutzung auf Servern.

Listing 2.1: Beispiel eines Dockerfiles für den Kompiler-Service

```
# Als Basis wird ein Scala/JDK Image genutzt
FROM sbtscala/scala-sbt:eclipse-temurin-21.0.7_6_1.11.2_3.7.1

# Arbeitsverzeichnis im Container erstellen
WORKDIR /app

# Notwendige Dateien fuer den Kompiler-Service kopieren
COPY check-api /app

# Befehl zum Starten des persistenten Kompiler-Dienstes
CMD ["sbt", "run"]
```

Docker-Container

Mit Docker-Images können beliebig viele Docker-Container erstellt werden. Dafür kommt der Befehl `docker run [Docker-Image Name]` zum Einsatz. Wird dieser Befehl ausgeführt, nutzt Docker die Informationen aus dem angegebenen Docker-Image, um einen Docker-Container zu erstellen. Bei der Erstellung des Containers wird eine neue Schicht erstellt, die auf die Daten des Docker-Images Lese-Zugriff hat und so die Informationen nutzen kann, ohne das Image zu verändern. In dem Container wird in unserem Listing-Beispiel 2.1 daraufhin die Befehlskette `sbt run` ausgeführt. Damit wird der Code aus dem Quellverzeichnis des Docker-Images kompiliert und ausgeführt. Somit wird unser Programmcode in einem eigenen Docker-Container ausgeführt. Der Docker-Container existiert dabei so lange, bis der Code beendet wird oder der Docker-Container gelöscht wird.

2.2.2. Isolation und Sicherheit (Sandboxing)

Ein Docker-Container trennt Prozesse in eigene, abgegrenzte Umgebungen. Diese Technologie erhöht zwar die Sicherheit, garantiert allerdings keine vollständig sichere Umgebung. Dafür müssten Sicherheitsanalysen und entsprechende Konfigurationen vorgenommen werden. Durch Docker-Container wird das System dennoch besser geschützt, als würde das gesamte System ohne Docker-Container ausgeführt werden. Da die Check-API zur Laufzeit Code ausführen muss, kann es passieren, dass dieser Code schadhaft ist. So könnte der auszuführende Code Befehle beinhalten, welche Betriebssystemfunktionen ausführen, die über die Anwendung hinausgehen. Dadurch könnten externe Prozesse oder Änderungen von Systemeinstellungen vorgenommen

werden. Dies wird eingegrenzt werden, wenn der Code in einem Docker-Container ausgeführt wird. In einem Docker-Container kann der Code zwar immer noch vollständigen Schaden anrichten, wie zum Beispiel den eigenen Docker-Container zu stoppen, dennoch wird es erschwert, auf außerhalb des Docker-Containers liegende Systeme zuzugreifen.

2.3. Orchestrierung mit Kubernetes

Docker-Container können, wie auch in unserem Fall, durch die Nutzung von Kubernetes (K8s) [The26a] noch weiter verbessert werden. Kubernetes sorgt sich um das System. Es hat einen Überblick über alle Container, die es verwaltet, und pflegt diese [The26c].

2.3.1. Grundlagen der Container Verwaltung

In K8s gibt es Cluster, Nodes und Pods. Cluster sind das Gesamtsystem. In ihnen wird alles verwaltet und beherbergt. Angefangen mit Nodes, welche in der Regel virtuelle Maschinen sind, die auf einem physischen System ausgeführt werden. Pods wiederum laufen auf diesen Nodes, wobei mehrere Pods in einer Node sein können. Als kleinste Instanz beinhalten die Pods schlussendlich Container. Einen oder auch mehrere Container, je nach System oder Notwendigkeit [The26b].

2.3.2. Hochverfügbarkeit und Self-Healing

Das System von K8s ist so aufgebaut, dass es Replicas ermöglicht. Replicas sind Kopien von Nodes oder Pods. Jedes Cluster besitzt einen Verteiler, genauer gesagt einen apiserver, welcher die Anfragen an die Nodes verteilt. Fällt eine Node aus, fällt dies dem Cluster auf. Daraufhin wird automatisch versucht, auf den anderen Nodes, sollte es mehrere in dem System geben, die Pods der ausgefallenen Node aufzubauen, um den Ausfall auszugleichen. Fällt ein einzelner Pod aus oder antwortet nicht mehr, wird auch dieser ersetzt oder so lange nicht mehr mit Anfragen versorgt, bis er wieder antwortet. Damit eine Node weiß, dass ein Pod ausgefallen ist, wird der Pod in regelmäßigen Intervallen von einem Kubelet auf ein Lebenszeichen angefragt. Antwortet der Pod innerhalb einer gewissen Zeit, weiß die Node, dass es dem Pod gut geht. Antwortet der Pod nicht, ist er entweder überlastet oder abgestürzt. Dieses System sorgt dafür, dass es sich selbst heilen kann, ohne komplett neu deployed oder manuell neu

gestartet werden muss [The26b]. Dieses Wissen reicht für diese Bachelorarbeit aus. Genauere Informationen, die das System in CodeTask genau funktioniert und aufgebaut ist, finden Sie in der Bachelorarbeit von Konstantin Leon Göke [Gök26].

3

Anforderungen

In diesem Kapitel wird auf das bereits bestehende Gesamtsystem und explizit auf das der Check-API, die zu dem Zeitpunkt bestehenden Probleme und Verbesserungsmöglichkeiten eingegangen.

3.1. Analyse des Altsystems und Problemstellung

Das Projekt CodeTask wird schon seit geraumer Zeit an der HTWG von Studenten entwickelt, ob für das Teamprojekt im sechsten Semester, für Bachelorarbeiten oder sogar Masterarbeiten an der Hochschule in Konstanz. Zu Beginn meiner Bachelorarbeit ist das Projekt schon sehr weit fortgeschritten, hat aber in vielen Bereichen noch Verbesserungspotenzial. So auch der Service Check-API.

Der Service Check-API hat dabei besonders kritische Schwachstellen. Zwar wird der Service schon in einem Docker-Container ausgeführt, um somit die Isolation zum restlichen System zu gewährleisten, allerdings gibt es keinerlei Sicherheiten, die dafür sorgen, dass der Service vor einer Abschaltung durch die interne Codeausführung geschützt ist. Ein weiteres Problem liegt darin, wie Code auf Maschinen ausgeführt wird. Eine endlose Schleife in einem Code kann die Verarbeitung innerhalb der Check-API blockieren und dadurch weitere Prüfungen verzögern oder verhindern. Das kann schon in allgemeinen Applikationen zu schwerwiegenden Problemen führen. Dies kann allerdings durch ausgiebige Tests leicht gefunden und frühzeitig behoben werden. In unserem Fall wissen wir allerdings nicht, ob der Code vom Nutzer sorgfältig geprüft wurde, und so können sich schnell unendliche Schleifen einschleichen. Zur Laufzeit kann

man schlecht prüfen, ob es sich in dem Code um eine unendliche Schleife handelt. Ein weiteres Problem liegt in der Art, wie die Aufgaben gestellt sind. Der Nutzer bekommt einen Code, wie in Listing 3.1 gezeigt. Dabei sieht der Student den öffentlichen Test, was dafür sorgt, dass der Student die Aufgabe geschickt umgehen kann und das eigentliche Ziel der Aufgabe nicht erfüllt. Schreibt der Student z. B. `val greeting = "hello world"`, wird der Test `greeting should be("hello world")` als korrekt validiert. Der Student hätte allerdings String-Konkatenation nutzen sollen, bei der der vorher definierte String `c` mit dem String `"_world"` zu einem String verbunden werden. Das Gleiche kann auch mit dem Wert `val b` gemacht werden, indem der Student `val b = 6` schreibt und nicht den eigentlich vorgegebenen mathematischen Term nutzt.

Listing 3.1: Beispiel-Code einer CodeTask Übungsaufgabe, ohne Nutzercode

```
//val a = 1
//val b = (? + ?) * 2
//val c =
//val greeting = c +

// --- Public Tests ---
a should be(1)
b should be(6)
c should be("hello")
greeting should be("hello world")
```

Des Weiteren bekommt der Nutzer ein sehr schwaches kompaktes Feedback durch den Service. Ist der Code des Nutzers korrekt, gibt der Check-API Service einen Statuscode von 200 OK aus. In einem System dieser Art sagt das aus, dass der Code korrekt geschrieben war und die Ergebnisse mit denen der Tests übereinstimmen. Ist der Code fehlerhaft, wird der Nutzer mit einem 400 Bad Request informiert, ohne eine weitere Nachricht. Dies ist unabhängig der Herkunft des Fehlers, heißt, ob es ein syntaktischer Fehler, also ein Fehler ist, der auf die Rechtschreibung des Codes zurückzuführen ist, oder ein falscher Wert ist, der dafür sorgt, dass einer der Tests fehlschlägt, ist dabei irrelevant. Ein 400 Bad Request Antwort-Code wird in der Regel für fehlerhafte Anfragen genutzt, kann dabei allerdings zusätzlich mit einer Nachricht versehen werden. Diese Nachricht enthält dabei typischerweise eine Begründung, warum die Anfrage fehlerhaft war, oder zumindest eine Möglichkeit, wenn das System selbst nicht ganz bestimmen kann, warum es zu einem Fehler gekommen ist.

3.2. Zielsetzung und Systemgrenzen

Das Ziel dieser Arbeit liegt darin, das System zu verbessern. Darunter zählt, eine erhöhte Sicherheit für das Gesamtsystem während der Codeausführung zu schaffen. Das System soll eine Möglichkeit anbieten, die dafür sorgt, dass der Code des Nutzers tiefergehend geprüft werden kann, sodass Lösungen über die öffentlichen Testfälle hinaus geprüft werden können. Der Service soll robuster gegen problematischen Nutzercode sein, sodass ein Ausfallen oder Abstürzen des Services vermieden wird. Des Weiteren soll der Service klarere Fehlermeldungen ausgeben, sollte mit dem Code des Nutzers etwas nicht stimmen. Als Letztes soll der Service auf Performance geprüft werden, um einschätzen zu können, wo die Grenzen des Services liegen und wie das System skaliert werden muss, um eine Nutzung an der Hochschule effizient und zuverlässig ermöglichen zu können. Diese Bachelorarbeit ist nicht dafür gedacht, die Inhalte der Aufgaben zu überarbeiten und zu erweitern. Diese Bachelorarbeit soll lediglich die Infrastruktur dafür erarbeiten.

3.3. Funktionale Anforderungen

Aus der zuvor beschriebenen Problemstellung und Zielsetzung leiten sich folgende funktionale Anforderungen zur Weiterentwicklung der Check-API ab.

FA1: Unterstützung versteckter Tests

Die Check-API muss versteckte Tests unterstützen. Dafür muss das System diese vor der Kompilierung mit den öffentlichen Tests und dem Nutzercode zusammenführen, sodass der gesamte Code in einem durch den Compiler geprüft werden kann.

FA2: Differenzierte Ergebnisauswertung

Die Check-API soll zu den bestehenden korrekten und fehlerhaften Aussagen zusätzlich Informationen liefern, worin das Problem bei einem fehlerhaften Ergebnis liegt. Dabei soll unterschieden werden, ob es sich um einen falsch geschriebenen Code, einen fehlgeschlagenen Test oder um eine Zeitüberschreitung handelt.

FA3: Strukturierte Rückmeldung

Die Aussagen zu fehlerhaften Anfragen sollen strukturiert in einem einheitlichen System über die vorhandene Schnittstelle geliefert werden.

FA4: Schutz von versteckten Tests

Es muss sichergestellt werden, dass zu keinem Zeitpunkt ein Nutzer die versteckten Tests sehen oder auslesen kann. Dazu gehört ebenfalls, dass Ausgaben des Compilers für die versteckten Tests nicht ohne Weiterverarbeitung an den Nutzer gelangen dürfen.

FA5: Begrenzung der Ausführungszeit

Braucht die Ausführung des Nutzercodes zu lange, muss die Ausführung abgebrochen werden. Eine zu lange Ausführungszeit deutet auf ineffizienten Code oder auf eine unendliche Schleife hin. In diesem Fall muss eine angepasste Meldung ausgegeben werden.

FA6: Behandlung kritischer Einreichungen

Fehlerhafter oder bösartiger Nutzercode darf die Ausführung nachfolgender Anfragen nicht behindern.

3.4. Nicht-funktionale Anforderungen

Weiterhin muss das System folgende Anforderungen an Sicherheit, Zuverlässigkeit und Leistungsfähigkeit erfüllen.

NFA1: Isolation der Codeausführung

Die Ausführung von Nutzercode darf nicht die restlichen Systeme gefährden oder kompromittieren. Der Code darf keinen Zugriff auf nicht benötigte Dateien, Systemressourcen oder interne Dienste erhalten.

NFA2: Vertraulichkeit interner Informationen

Versteckte Tests, Dateipfade, Stacktraces oder weitere interne Informationen dürfen nicht ungefiltert an den Nutzer ausgegeben werden.

NFA3: Robustheit

Die Verfügbarkeit der Check-API darf nicht durch absichtlich problematische oder nicht terminierende Einreichungen beeinträchtigt werden.

NFA4: Leistungsfähigkeit

Der Service Check-API muss auch unter Last, verursacht durch mehrere simultane Anfragen, innerhalb akzeptabler Antwortzeiten reagieren. Die genauen Grenzen werden in dem Kapitel 6 Evaluation untersucht.

4

Konzept und Architektur

Dieses Kapitel erläutert den konzeptionellen Aufbau der erweiterten Check-API. Dafür werden die Anforderungen aus Kapitel 3 in die Zielarchitektur, den Ablauf einer Anfrage und in die Sicherheitsmaßnahmen übersetzt.

4.1. Überblick über die Zielarchitektur

Die Zielarchitektur baut dabei auf der bereits bestehenden isolierten Check-API sowie den bestehenden Systemen wie der Haupt-API, Frontend und Datenbank auf. Die Haupt-API ist für die Verwaltung aller Anfragen sowie der Zusammensetzung notwendiger Daten zuständig. Für die Prüfung des Nutzercodes ruft sie die Check-API auf und übergibt die notwendigen Informationen, wie die öffentlichen Tests, Nutzercode und die neu eingeplanten, versteckten Tests.

Die Check-API übernimmt, separiert vom restlichen System, den Prüfdienst, die Verarbeitung der Einreichung und liefert ein Ergebnis an die Haupt-API zurück. Die Weiterentwicklung im Rahmen dieser Bachelorarbeit setzt primär hier an und erweitert das System um eine abgesicherte, versteckte Test Verarbeitung, eine Überprüfung auf bösartige Codeabschnitte und eine aussagekräftige Ergebnissrückgabe.

Abbildung 4.1 zeigt den konzeptionellen Aufbau der Zielarchitektur. Die Haupt-API sammelt die Daten und leitet sie an die Check-API weiter. Somit bleibt die Haupt-API für die Verwaltung und die Check-API ist weiterhin für die Prüfung zuständig.

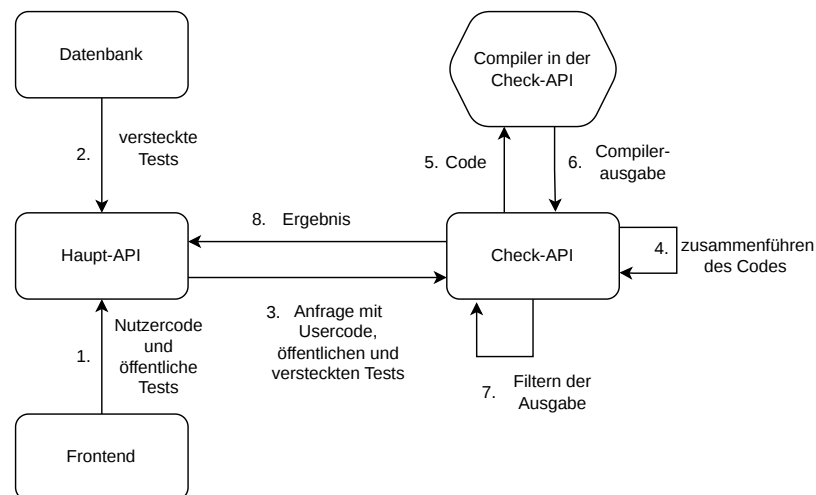


Abbildung 4.1: Konzept-Architektur

4.2. Verarbeitungsablauf einer Einreichung

Zu Beginn der Verarbeitung schickt die Haupt-API eine Anfrage an die Check-API, in dieser Anfrage sind bereits die öffentlichen, versteckten Tests und der Nutzercode enthalten. Die Check-API nimmt diese Daten entgegen, verarbeitet diese und vereint sie für eine einzige Ausführung. So können, wie bisher, die öffentlichen Tests, Nutzercode und mit der Änderung auch die versteckten Tests gemeinsam kompiliert und ausgeführt werden. Das ist wichtig, damit beide Testsysteme mit den entsprechenden Daten arbeiten können, ohne dass der gleiche Nutzercode zwei Mal ausgeführt werden muss. Tritt bei der Kompilierung oder der Ausführung ein Fehler auf, wird die Codeausführung terminiert, die Ergebnisse des Compilers von der Check-API überprüft und relevante Informationen zurück an die Haupt-API übergeben. Zeitgleich mit der Ausführung des Nutzercodes in der Check-API überwacht die Check-API, wie lange eine Ausführung dauert. Dauert diese zu lange, wird die Ausführung beendet und die Check-API antwortet der Haupt-API mit einer entsprechenden Nachricht.

4.3. Konzept zur Integration versteckter Tests

Versteckte Tests haben den Nutzen, zu prüfen, ob der Nutzer die Aufgabe auf die vorgesehene Art und Weise löst. Versteckte Tests gehen dabei über öffentliche Tests hinaus, sie sind dem Nutzer verborgen, sodass eine gezielte Anpassung an diese Testfälle er-

schwert wird. Das Problem mit öffentlichen Tests ist, dass sie vom Nutzer gesehen werden. Genau das ist auch der Sinn von öffentlichen Tests. Dadurch, dass der Nutzer sie allerdings sehen kann, kann der Nutzer den Test auch einfach umgehen, wie schon in der Analyse des Altsystems und Problemstellung 3.1 erklärt. Versteckte Tests umgehen dies, da der Nutzer nicht sieht, was diese Tests genau prüfen. Damit die versteckten Tests versteckt bleiben, werden diese nicht wie die öffentlichen Tests beim Aufrufen einer Aufgabe ans Frontend übergeben, sondern werden erst hinzugezogen, sobald der Nutzer seine Aufgabe abgibt. In diesem Moment wird der Nutzercode zusammen mit den öffentlichen Tests und weiteren Informationen an die Haupt-API geschickt. Die Haupt-API kann mit den zusätzlichen Informationen die versteckten Tests in der Datenbank anfragen und so gemeinsam alle drei Bestandteile an die Check-API überreichen. So wird sichergestellt, dass die genauen Testdaten der versteckten Tests nicht im Frontend einsehbar sind. Des Weiteren muss sichergestellt werden, dass bei der Ausführung des Codes in der Check-API im Falle eines Anschlagens eines versteckten Tests die Ausgaben des Stacktraces abgefangen werden. Die Ausgaben des Stacktraces beinhalten genaue Details darüber, warum der Test fehlgeschlagen ist. Würden diese Informationen an den Nutzer gelangen, könnte dieser mithilfe der Informationen die Tests präzise umgehen. Das würde den Nutzen von versteckten Tests nichtig machen. Anstelle des Stacktraces soll dem Nutzer eine generische Nachricht übergeben werden, mit welcher der Nutzer versteht, was falsch ist, ohne genau zu wissen, wie er den Test umgehen kann.

4.4. Konzept zur isolierten Codeausführung

Zur Isolierung gehören mehrere kritische Bausteine. Angefangen mit einer Abkapselung des Compilers vom restlichen System. Diese Abkapselung besteht bereits durch die Check-API als eigenen Service. Im Rahmen dieser Arbeit wird diese bestehende Trennung durch zusätzliche Schutz- und Stabilitätsmechanismen erweitert. Diese Abkapselung ist kritisch, da sie dafür sorgt, dass im Falle eines böartigen Nutzercodes nicht das gesamte System, sondern nur dieser Service beeinträchtigt wird. Fällt der Service aus, bleibt die Hauptanwendung grundsätzlich erreichbar, die Prüfung von Nutzercode ist jedoch beeinträchtigt. Ausschließlich dieser Service kann nicht mehr erreicht werden. Natürlich ist ein Ausfall eines Services dennoch problematisch. Würde ein ausgefallener Service keinen Unterschied machen, wäre dieser Service schlichtweg irrelevant. Die Check-API hat allerdings eine der wichtigsten Aufgaben in diesem System. Wie ein möglicher Ausfall des Services unproblematischer gemacht werden kann,

wird in Kapitel 5 Implementierung genauer erläutert. Ein weiterer Schritt der Isolierung ist, den Service robust gegen die Ursachen eines Ausfalls zu machen. Die größte Schwäche der Check-API ist, dass bösartiger Code sie ausschalten kann. Daher muss der auszuführende Nutzercode auf kritische Inhalte untersucht werden. Sollte solch ein kritischer Inhalt entdeckt werden, darf die Ausführung nicht stattfinden. Die Anfrage muss unterbrochen werden und mit einer entsprechenden Nachricht geantwortet werden. Zusätzlich ist in der Isolierung die Abgrenzung von Ressourcen relevant. Im Kontext der Check-API bedeutet das zum einen, dass die Check-API selbst keinerlei Zugriff auf Dateien oder Informationen der anderen Services haben darf. Andere Services nutzen kritische Daten wie Zugangsinformationen für die Datenbank oder API-Schlüssel externer Services. Teilt sich die Check-API ein nicht isoliertes System mit solchen anderen Services, kann sich bösartiger Nutzercode Zugriff zu genau diesen Informationen verschaffen. Wie diese Isolierung im Detail erreicht wird, wird im Kapitel 5 Implementierung genauer aufgeschlüsselt.

4.5. Konzept zur Ergebnis- und Fehlerbehandlung

Die Fehlerbehandlung ist für eine Programmier-Lernplattform eines der entscheidendsten Elemente, um einen erfolgreichen Lernfortschritt zu gewährleisten. Ohne ein klares Ergebnis weiß ein Nutzer nicht, ob sein Code richtig oder falsch war. Aber hier hört es nicht auf. Der Nutzer muss auch wissen, warum etwas nicht erfolgreich war. Hier muss unterschieden werden, welche Fehler es geben kann und wie mit diesen umgegangen werden muss. In CodeTask müssen 6 Fälle behandelt werden, um sicherzustellen, dass der Nutzer das best mögliche Ergebnis mitgeteilt bekommt. Angefangen mit bösartigem Nutzercode, welcher noch vor der Kompilierung abgefangen werden muss, um eine Ausführung zu verhindern, was anderenfalls zu kritischen Fehlern führen kann. In solch einem Fall muss mit einer Aussage geantwortet werden, welche dem Nutzer nicht mitteilt, was exakt im Code diesen Fehler ausgelöst hat, sondern lediglich, dass die Ausführung verboten ist. Der zweite Fall tritt auf, sollte der Nutzercode Fehler beinhalten, welche dafür sorgen, dass der Compiler den Code nicht ordnungsgemäß kompilieren kann. In diesem Fall kann der vom Compiler selbst bereitgestellte Fehler dem Nutzer übergeben werden. Der dritte Fall tritt auf, sollte es bei der Ausführung nach der Kompilierung zu Fehlern kommen. Bei öffentlichen Tests und Laufzeitfehlern können detaillierte Informationen ausgegeben werden. Diese sollten dennoch überprüft werden. Fall vier und fünf entstehen, wenn der öffentliche oder der versteckte Test anschlägt. Bei versteckten Tests muss das System die Ausgabe ab-

fangen, um die Geheimhaltung des Tests zu gewährleisten. Stattdessen muss auch hier dem Nutzer eine generische Fehlermeldung zurückgegeben werden. Der letzte Fall ist die Zeitüberschreitung, sollte der Nutzercode zu lange ausgeführt werden. In diesem Fall wird die Ausführung nach einer vordefinierten Zeit abgebrochen. Darüber muss der Nutzer allerdings informiert werden. Auch hier kommt eine generische Antwort zum Einsatz, da es unterschiedliche Gründe geben kann, warum der Nutzercode so lange in der Ausführung braucht.

5

Implementierung

5.1. Ausgangszustand der Check-API und Aufgabenverarbeitung

Damit verständlich wird, wie die Änderungen durch diese Bachelorarbeit das System verändern, gehe ich in diesem Abschnitt darauf ein, wie das System im nahen Umfeld der Änderungen funktioniert und aufgebaut ist.

5.1.1. Übungsaufgabenverwaltung

In diesem Unterabschnitt gehe ich genauer auf die Bereitstellung, Verarbeitung und Speicherung von Übungsaufgaben ein.

Koan-Repository

Das Koan-Repository ist ein Git-Repository, in welchem sämtliche Übungsaufgaben abgelegt sind. Diese Aufgaben können von Nutzern mit der Rolle Professor über das Frontend geladen werden. Dabei wird mit spezieller Berechtigung das Repository angefragt und CodeTask erhält die Aufgaben in der Haupt-API.

Koan-Parser

Die Aufgaben werden daraufhin in speziell festgelegte Elemente zerteilt. Dafür zuständig ist der Parser, dieser besteht aus Reg-Ex Regeln, welche ausdrücken, wo ein Element anfängt, aufhört und was dazwischen relevant ist.

Datenbank

Die aufgeteilten Aufgaben werden daraufhin in der Datenbank gespeichert. Das sorgt dafür, dass die Aufgaben nicht für jede Anfrage aus dem Repository geladen und durch den Parser verarbeitet werden müssen.

Frontend

Öffnet ein Nutzer eine Lektion und wählt darin eine Aufgabe, welche er lösen möchte, so wird von der Haupt-API die korrekte Aufgabe geladen und mit allen Informationen im Frontend dargestellt.

5.2. Erweiterung des Parsers und der Testdatenverarbeitung

Anfangen mit der Implementierung der Infrastruktur für die versteckten Tests, mussten Änderungen an dem Parser vorgenommen werden. Für eine saubere Aufteilung musste zunächst verstanden werden, wie der Parser bisher gearbeitet hat. In dem Listing 5.1 wird die CodeTask Aufgabe von einem Koan abgebildet. Dieser Aufgabenblock ist speziell formatiert und nutzt spezielle Markierungen wie `//solve`, `//endsolve`, `//test` und `//endtest`. Mithilfe dieser Markierungen kann der Parser gezielt nach diesen Bereichen suchen und diese an diesen Stellen trennen. Alles, was vor dem `//solve` steht, dient als Hilfestellung für den Nutzer und wird ohne weiteres übernommen. Der Inhalt in dem `solve` Block ist die Musterlösung für diese Aufgabe und muss getrennt werden, damit diese nicht von Anfang an dem Nutzer angezeigt wird. In dem `test` Block ist der öffentliche Test definiert und soll dem Nutzer ebenfalls als Hilfestellung angezeigt werden. Der Parser hat aus diesen Blöcken `code`, und `codeSolution` gemacht. Hier fehlen aber die Tests. Das Feld `code` wurde im Parser wie im Listing 5.2 gezeigt zusammengesetzt. Dabei wird der erste Bereich übernommen, der `solve` Block entfernt und durch ein `_#_` ersetzt und zum Schluss wird der `test` Block wieder angehängt. Das Feld `codeSolution` besteht aus dem `solve` Block. Diese Lösung ist im Prinzip nicht schlecht, reicht für versteckte Tests aber nicht mehr aus. Anstelle das `code` Feld auch die Tests enthält, wird jetzt nur noch der Code vor dem `//solve` zusammen mit dem `_#_` gespeichert. Neu dazu kommen die Felder `tests` und `hiddenTests`. Der Parser speichert also jeden Block einzeln ab. Das erleichtert bzw. ermöglicht die spätere Bereitstellung der relevanten Daten. Der Parser wurde so erweitert, dass er die Möglichkeit hat, versteckte Tests zu erkennen. Listing 5.3 zeigt die Erweiterung des in Listing 5.1 dargestellten Koans. Koans können um einen `//hiddentest` und `//enhiddentest` Block erweitert werden und können ohne weitere Anpassung des Parsers verarbeitet werden.

Listing 5.1: Koan Ausschnitt ohne versteckte Tests

```
codetask(
  """# Übung: Mehrzeilige Adresse mit Interpolation

  Nutze einen mehrzeiligen interpolierten String mit 'stripMargin', um eine
  Adresse aufzubauen.
  """
) {
  // val name = "HIERÄNDERN"
  // val street = "123 Navy St"
  // val city = "HIERÄNDERN"
  // val address =
  // s"""
  //     |$HIERÄNDERN
  //     |$HIERÄNDERN
  //     |$HIERÄNDERN
  // """
  // solve
  val name = "Grace Hopper"
  val street = "123 Navy St"
  val city = "Arlington, VA 22201"
  val address = s"""
    |$name
    |$street
    |$city
  """.stripMargin
  //endsolve

  //test
  address should include ("Grace Hopper")
  address should include ("123 Navy St")
  address should include ("Arlington")
  //endtest
}
```

Listing 5.2: Das Feld "code" nach dem Bestehenden Parser

```
code=
  """// val name = "HIERÄNDERN"
  // val street = "123 Navy St"
  // val city = "HIERÄNDERN"
  // val address =
  // s"""
  //     |$HIERÄNDERN
  //     |$HIERÄNDERN
  //     |$HIERÄNDERN
  // """
  // stripMargin
  _#_
  address should include ("Grace Hopper")
  address should include ("123 Navy St")
  address should include ("Arlington")"""
```

Listing 5.3: Koan Ausschnitt mit versteckten Tests

```
...
//test
address should include ("Grace Hopper")
address should include ("123 Navy St")
address should include ("Arlington")
//endtest

//hiddentest
address.contains("\n") shouldBe true
val lines = address.linesIterator.toList
lines.length shouldBe 3
lines.mkString(" ") should include (name)
lines.mkString(" ") should include (street)
lines.mkString(" ") should include (city)
//endhiddentest
```

5.3. Bereitstellung und Zuordnung von versteckten Tests

Wie gerade schon angesprochen, werden mit den Veränderungen im Parser die Blöcke in einzelne Felder abgelegt und über die bestehende Datenbankverbindung in die Datenbank geschrieben. Für die Speicherung in der Datenbank müssen lediglich die neuen Felder übergeben werden. Die beschriebene Änderung erlaubt dem System, eine getrennte Übertragung der einzelnen Blöcke. Die Musterlösung und der Code, welcher die Aufgabenstellung helfend unterstützt, muss zusammen mit den öffentlichen Tests über die Haupt-API an das Frontend geschickt werden. Hier muss beachtet werden, dass die versteckten Tests nicht ebenfalls mitgeschickt werden, da diese sonst ausgelesen werden können. Löst der Nutzer die Aufgabe und schickt sein Ergebnis ab, so wird zusammen mit dem Nutzercode, die öffentlichen Tests, die Lösung und Informationen über die Aufgabe selbst mitgeschickt. Diese Informationen werden in der Haupt-API genutzt, um die korrekten, versteckten Tests in der Datenbank zu finden, sodass der Nutzercode zusammen mit den öffentlichen und versteckten Tests an die Check-API übergeben werden kann.

5.4. Anpassung der Check-API

Unter die Anpassungen der Check-API fallen eine Erweiterung der Schnittstelle zur Unterstützung von versteckten Tests, eine erhöhte Sicherheit und Isolation, Verbesserungen der Leistung sowie eine verbesserte Rückmeldung für den Nutzer. Die Check-API muss die Anfrage von der Haupt-API mit den neuen Änderungen auch entsprechend

entgegennehmen. Da von der Haupt-API die Felder `code`, `tests` und `hiddenTests` übergeben werden, müssen diese auch in der Check-API angenommen und verarbeitet werden. In der Check-API angekommen, werden die Felder an den sogenannten `codeTester` übergeben. Hier werden die drei Felder erstmals wieder zu einem großen, vereinten Stück Code in Form von einem String. Dieser String wird mit einer Liste voller verbotener Schlüsselwörter geprüft. Sollte auch nur eines dieser Schlüsselwörter in dem Code vorkommen, wird die Ausführung sofort mit einem Failure und einer Exception abgebrochen. Die Prüfung verhindert ausgewählte, bekannte Problemfälle, ersetzt jedoch keine vollständige Sicherheitsabsicherung der Codeausführung. Dazu gehört z. B. `System.exit`. Dieser Befehl schaltet den gesamten Prozess aus, in dem er läuft. Nach dieser Sicherheitsüberprüfung werden die einzelnen Blöcke leicht formatiert und in ein vordefiniertes Template eingesetzt. Dieses Template beinhaltet alle notwendigen Imports sowie Methoden, um die Tests korrekt ausführen zu können. Das befüllte Template ist ab diesem Zeitpunkt vollständig und kann kompiliert werden. Dafür wird das Template in eine virtuelle Datei geschrieben. Eine virtuelle Datei existiert dabei nur im Arbeitsspeicher und wird nicht auf die Festplatte geschrieben. Das ermöglicht schnellere Geschwindigkeiten. Die Idee ist nicht neu, sondern war bereits in dem vorhandenen Kompiler eingebaut. Ist diese virtuelle Datei erstellt, kann der Kompiler gestartet werden. Der Kompiler übersetzt den Code, sodass der Code ausgeführt werden kann. Der übersetzte Code wird temporär auf die Festplatte geschrieben. Das bringt den Vorteil, dass die Codeausführung mehrerer Anfragen parallel erfolgen kann und so eine schnellere Antwortzeit auch unter Last ermöglicht wird. Die Parallelität ist dabei allerdings von der Hardware begrenzt. Würden pro Anfrage dauerhaft, ungebremst neue Unterprozesse erstellt werden, würde das System sich selbst verlangsamen, da mit der Erstellung eines Unterprozesses ein gewisser Aufwand verbunden ist. Daher begrenzen wir die Anzahl der gleichzeitig existierenden Unterprozesse. Mindestens wird ein Unterprozess erstellt, maximal aber $Anzahl = \frac{CPU-Kerne}{2}$. Das führt dazu, dass wir nicht die gesamte Leistung beanspruchen und noch genügend Leistung für das restliche System besteht. In dem neu erstellten Unterprozess wird der auf der Festplatte gespeicherte, kompilierte Code ausgeführt. Diese Dateien müssen nach dem Ausführen zum Schluss noch aufgeräumt werden. Damit ist der grobe Ablauf und die Implementierung in Teilen beschrieben. Etwas tiefer wird auf die Leistungsoptimierung noch in Kapitel 6 Evaluation eingegangen.

5.5. Zeitüberschreitung

Die Check-API wurde noch in weiteren Bereichen verbessert, darunter das Handhaben von Zeitüberschreitungen. Direkt nachdem der Unterprozess, zuständig für das Ausführen des Codes, gestartet ist, wird ein Thread gestartet, dessen einziger Job es ist, den Unterprozess zu überwachen. Der Thread wartet blockierend, bis der Unterprozess beendet ist. Das passiert entweder durch einen Laufzeitfehler oder durch das erfolgreiche Erreichen des Endes vom Code. Solange der Thread blockierend wartet, wartet ebenfalls der Hauptprozess, entweder darauf, dass der Thread ein Ergebnis bekommen hat, oder eine gewisse Zeit abgelaufen ist. Aktuell 5 Sekunden, dies ist allerdings frei anpassbar, sollte dieses Zeitfenster zu eng gewählt worden sein. Tritt der Fall ein, dass der Hauptprozess diese Zeit vollständig warten musste, greift er ein und beendet den Unterprozess, der gerade den Code ausführt. Dieses Verhalten sorgt dafür, dass fehlerhafter Code nicht unendlich lange laufen kann und permanent Ressourcen verbraucht. Solch eine lange Laufzeit kann mehrere Ursachen haben. Darunter eine klassische unendliche Schleife, welche sich für immer wiederholt. Das Ergebnis ist, der Code wird für eine unendliche Dauer ausgeführt. Eine zweite Ursache kann unoptimierter Code sein, der sehr viele unterschiedliche Fälle durchlaufen muss und so sehr lange in der Ausführung braucht. Dieser Fall kann problematisch sein, da dieser Code eventuell sogar komplett korrekt sein kann und dennoch abgebrochen wird. Im Fall von CodeTask stellt das allerdings kein besonderes Problem dar, da die Aufgaben zum Lernen recht klein gewählt werden und keine extrem komplexen Algorithmen mit tausenden Daten fordern. Sollte es dennoch zu diesem Problem kommen, kann das Zeitlimit angepasst werden.

5.6. Fehlerbehandlung und Strukturierte Ergebnissrückgabe

Im Vergleich zum ursprünglichen System wurde die Check-API ebenfalls um einige Fehlerbehandlungen und Ergebnissrückgaben erweitert. Ursprünglich hat die Check-API lediglich mit zwei unterschiedlichen Fällen geantwortet. Einmal mit einem 400 Bad Request und einmal mit einem 200 OK Statuscode. Bei einem 400 Bad Request wurde dabei eine Nachricht mitgeschickt, welche vermuten lässt, dass versucht wurde ein Compiler oder Laufzeitfehler mitschicken zu können, hier ist allerdings etwas schief gegangen, sodass lediglich immer die Fehlermeldung `testing.Testing$` im Frontend angezeigt wurde. Die Ursache dafür war, dass lediglich der geworfene Fehler weiter gereicht wurde und nicht tiefer in das Objekt eingegangen wurde. In der überarbeiteten

Version wird die gesamte Fehlermeldung, wie in Tabelle 5.1 visualisiert, umgesetzt. Dabei wird ein einheitliches JSON, bestehend aus `Fehlercode`, `Status`, `errorType`, also in welche Kategorie der Fehler fällt, und die `message`, also die Nachricht, welche erklärt, warum es möglicherweise zu diesem Fehler gekommen ist. Dieses System sorgt für eine wesentlich klarere Aufschlüsselung des Fehlers und macht es dem Nutzer im Frontend wesentlich einfacher, zu erkennen, wo das Problem liegen könnte. Der Wert `errorType` beschreibt die grobe Fehlerkategorie. Die konkrete Unterscheidung einzelner Ausführungsfehler erfolgt zusätzlich über die interne Fallbehandlung und die zurückgegebene Nachricht.

Tabelle 5.1: Fehlerbehandlung und Rückgabemodell der Check-API

Fehlerfall	errorType	Rückgabe
Erfolgreiche Prüfung	–	200 OK mit {"status": "success"}
Fehlender Request-Body	–	400 Bad Request mit {"status": "error", "message": "Missing code to check."}
Sicherheitsverstoß	security	400 Bad Request mit {"status": "error", "errorType": "security", "message": "Security Violation: Forbidden keyword detected."}
Kompilierfehler	compilation	400 Bad Request mit {"status": "error", "errorType": "compilation", "message": "Compilation Error:..."}
Fehlgeschlagener öffentlicher Test	execution	400 Bad Request mit {"status": "error", "errorType": "execution", "message": "..."}
Fehlgeschlagener versteckter Test	execution	400 Bad Request mit {"status": "error", "errorType": "execution", "message": "Hidden Requirement not met:} Your solution is not generic enough (e.g., hardcoded values detected)."}
Timeout / Endlos-schleife	execution	400 Bad Request mit {"status": "error", "errorType": "execution", "message": "Execution Error: Timeout (Infinite loop detected?)"}
Sonstiger Laufzeitfehler	execution	400 Bad Request mit {"status": "error", "errorType": "execution", "message": "..."}

6

Evaluation

6.1. Ziel der Evaluation

Ziel der Evaluation ist es, zu überprüfen, ob die in Kapitel 3 definierten Anforderungen erfüllt worden sind. Dafür werden Ergebnisse verschiedener Anfragen aus dem Frontend und Performance Messungen zur Auswertung der Check-API unter Last in unterschiedlichen Umgebungen ausgewertet und verglichen.

6.2. Evaluationsumgebung

Es gibt drei unterschiedliche Umgebungen, die für Tests genutzt wurden. Angefangen mit der lokalen Umgebung auf meinem eigenen Laptop mit 16GB Arbeitsspeicher und dem M1 Pro Chip mit 6 Leistungs- und 2 Effizienzkernen wurde die Check-API einzeln über das Docker-Image lokal ausgeführt. Die zweite Umgebung ist die aktuelle Live-Version, welche zu diesem Zeitpunkt aus dem Deployment mit Docker-Compose besteht und somit alle Services in eigenen Containern laufen. Eine genaue Information über Systemressourcen steht nicht zur Verfügung. Die dritte Umgebung ist eine Kubernetes Umgebung, welche im Rahmen der Bachelorarbeit Von Docker Compose zu Kubernetes: Evaluation der Zuverlässigkeit [Gök26] von Konstantin Göke zeitgleich zu dieser entwickelt wurde. Wie auch in der Docker-Compose Umgebung ist bei der Kubernetes Umgebung keine Information über die Hardware bekannt. Die Kubernetes Umgebung nutzt für die Check-API bis zu 4 Pods, welche jeweils bis zu 2000m

CPU nutzen können. Für die Leistungsüberprüfung wurde das Tool Gatling [Gat26] für Scala genutzt, mit welchem Testfälle und Metriken definiert werden können. Damit die Vergleiche möglichst realistisch und vergleichbar bleiben, wurden zwei Endpunkte in den Systemen genutzt. Es wurde speziell ein Endpunkt in der Haupt-API eingebaut, mit welchem direkt die Check-API über den gleichen Weg wie auch im normalen Betrieb aufgerufen werden kann. Das wurde notwendig, da die Kubernetes Umgebung die Check-API nicht ohne weiteres nach außen erreichbar macht. Mehr dazu im Abschnitt 6.4. Die Tests schicken je nach Endpunkt perfekt abgeschnittene Anfragen an den jeweiligen Endpunkt. Gatling verfügt dabei über die Einstellung, wie viele Nutzer es über einen gewissen Zeitraum geben soll. Ein Nutzer schickt dabei immer alle im Test vorhandenen Anfragen ab, wartet auf eine Antwort und schaltet sich ab. Mit der Codezeile `constantConcurrentUsers(20).during(2.minutes)` werden so z. B. durch Gatling versucht, zwei Minuten lang konstant 20 Nutzer gleichzeitig zu halten. Wird ein Nutzer fertig, startet Gatling direkt einen neuen Nutzer, der erneut den gleichen Prozess durchläuft. Gatling speichert dabei unterschiedliche Werte und erstellt nach Fertigstellung des Tests eine Zusammenfassung mit all diesen Werten. Auf diese Werte wird sowohl im Abschnitt 6.4 und 6.5 eingegangen.

6.3. Funktionale Evaluation

Wie die Tabelle 6.1 zeigt, werden die einzelnen Fehlerfälle unterschieden und in dem gewünschten Format ausgegeben. Besonders relevant ist die Ausgabe über verbotene Schlüsselwörter, da dies zeigt, dass bekannte, kritische Befehle erkannt und vor der Ausführung abgefangen werden. Auch die versteckten Tests schlagen an und werden dabei in dem gewünschten generischen Format ausgegeben. Für jeden Fehlerfall wurde im Frontend entsprechender Code geschrieben, abgeschickt und mit dem erwarteten Fehlercode und dem Inhalt der Rückmeldung verglichen.

6.4. Evaluation der Robustheit und Verfügbarkeit

Wie schon in dem vorherigen Absatz 6.3 aufgezeigt, wird die Robustheit leicht verbessert durch die Filterung von bössartigen Befehlen. Die Analyse der unterschiedlichen Systeme hat dabei aufgezeigt, dass in der Docker-Compose Umgebung ein gravierendes Problem besteht. Sollte der Check-API-Container abstürzen, ist das System zwar darauf ausgelegt, den Container erneut zu starten, dabei geht allerdings etwas schief,

Tabelle 6.1: Funktionale Testfälle der Check-API

Testfall	Erwartetes Verhalten	Ergebnis
Korrekte Einreichung	Rückgabe von 200 OK	Erfolgreich
Kompilierungsfehler	Fehlerklassifikation compilation	Erfolgreich
Fehlgeschlagener öffentlicher Test	Ausgabe einer verständlichen Fehlermeldung	Erfolgreich
Fehlgeschlagener Hidden Test	Generische Fehlermeldung ohne Testdetails	Erfolgreich
Endlosschleife	Abbruch nach Timeout	Erfolgreich
Verbotenes Schlüsselwort	Abbruch vor der Ausführung	Erfolgreich

sodass der Container nicht erneut starten kann. Dieses Problem besteht bei der Kubernetes Umgebung allerdings nicht. Fällt hier einer der Check-API-Pods aus, so wird direkt ein neuer Pod gestartet und kann den Ausfall schnellstmöglich wieder ausgleichen. Kubernetes bietet des Weiteren den Vorteil, auf unterschiedliche Server verteilt und repliziert zu werden, was im Vergleich zu der Docker-Compose Umgebung einen Single Point of Failure verhindern kann.

6.5. Performance-Evaluation

In diesem Abschnitt werden die Ergebnisse der Performance Tests aufgeschlüsselt. Dafür wurden die lokale, Docker-Compose und die Kubernetes Umgebung verglichen. Die Tests bestanden aus einer einfachen Anfrage, welche perfekt auf die Endpunkte in der Haupt-API und der Check-API abgestimmt wurden, um möglichst einheitliche Testergebnisse zu erzielen. Dabei wurden die Systeme einmal mit 50 dauerhaften Nutzern über zwei Minuten und einmal mit 100 dauerhaften Nutzern über zwei Minuten getestet. In den Abbildungen 6.1, 6.2 und 6.3 wurde die Antwortzeit der optimierten mit der ursprünglichen Version verglichen. Da ich von der Docker-Compose Umgebung vor der Verbesserung keinen Test ausgeführt hatte, bestehen hier leider keine Daten. Die Messungen deuten auf eine Verbesserung hin. Aufgrund der unterschiedlichen Testum-

gebungen und teilweise unbekannten Hardware sind die Ergebnisse jedoch nur eingeschränkt direkt vergleichbar. In der Kubernetes Umgebung wird eine Verbesserung von 33% erzielt. Das entspricht über 15 Sekunden Unterschied. Lokal kann das System sogar eine Verbesserung von fast 70% erreichen, was über 19 Sekunden Differenz entspricht. Lokal muss man allerdings beachten, dass es keine Übertragungslatenz zu einem Server gibt und die Kubernetes Umgebung die Anfragen nicht direkt in der Check-API, sondern in der Haupt-API erhält, da die Check-API ohne weiteres nicht von außerhalb des Systems erreichbar ist. Dadurch muss ebenfalls etwas mehr Rechenzeit für die Verarbeitung und Weiterleitung der Anfrage berechnet werden. Ebenfalls kann man in den Diagrammen erkennen, dass die Docker-Umgebung immer etwas schneller antwortet als die Kubernetes Umgebung. In der Abbildung 6.4 wird die Antwortzeit der optimierten Version zwischen Docker-Compose und Kubernetes mit 50 und 100 Nutzern verglichen. Hier wird sichtbar, dass 50 Nutzer und damit weniger Last auf dem System deutlich bessere Ergebnisse in der Antwortzeit liefern als 100. Die Tests sollen offenbaren, wo die Grenzen des Systems liegen und wie eine realistische Nutzung unter extremen Bedingungen, wie einer Klausur, aussieht.

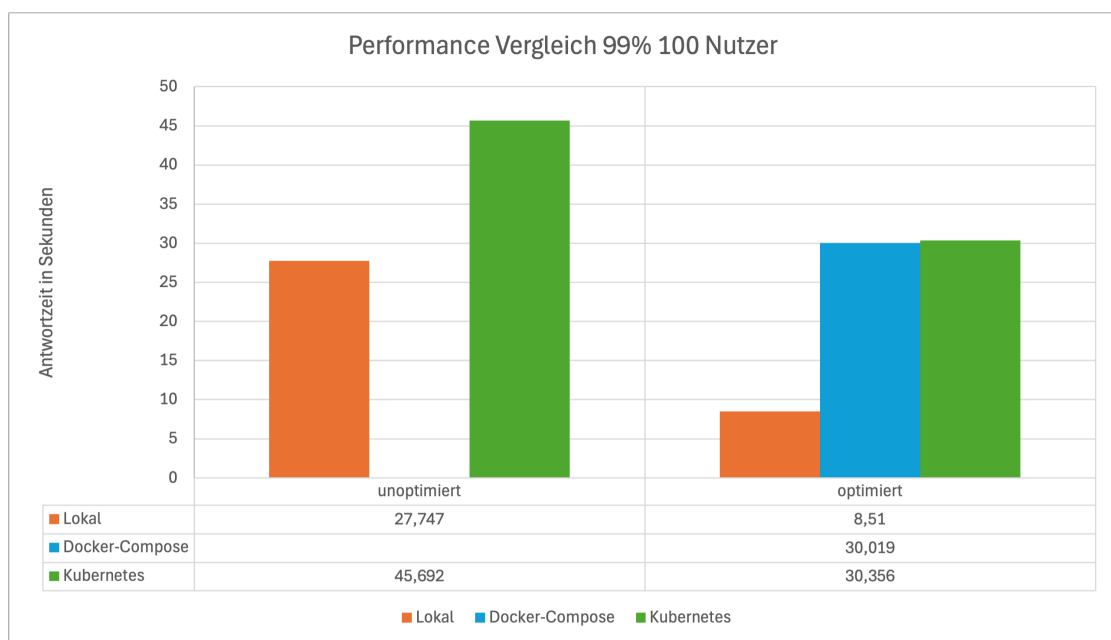


Abbildung 6.1: Performance Vergleich zwischen der lokalen, Docker-Compose und der Kubernetes Umgebung mit jeweils 100 gleichzeitigen Nutzern, im Vergleich die 99. Perzentil der Antwortzeit

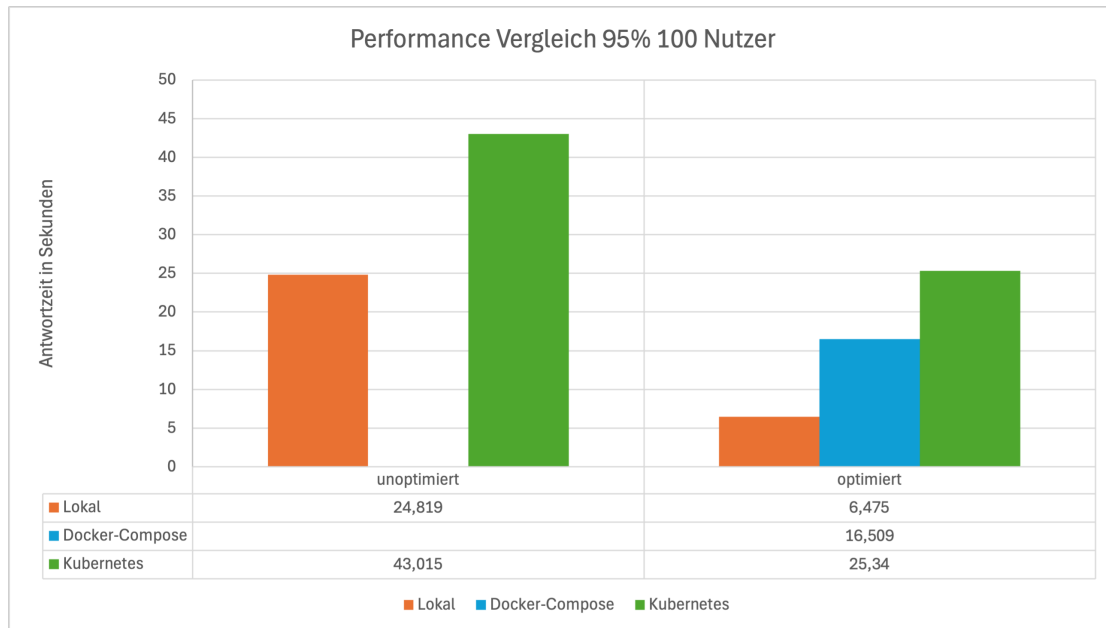


Abbildung 6.2: Performance Vergleich zwischen der lokalen, Docker-Compose und der Kubernetes Umgebung mit jeweils 100 gleichzeitigen Nutzern, im Vergleich die 95. Perzentil der Antwortzeit

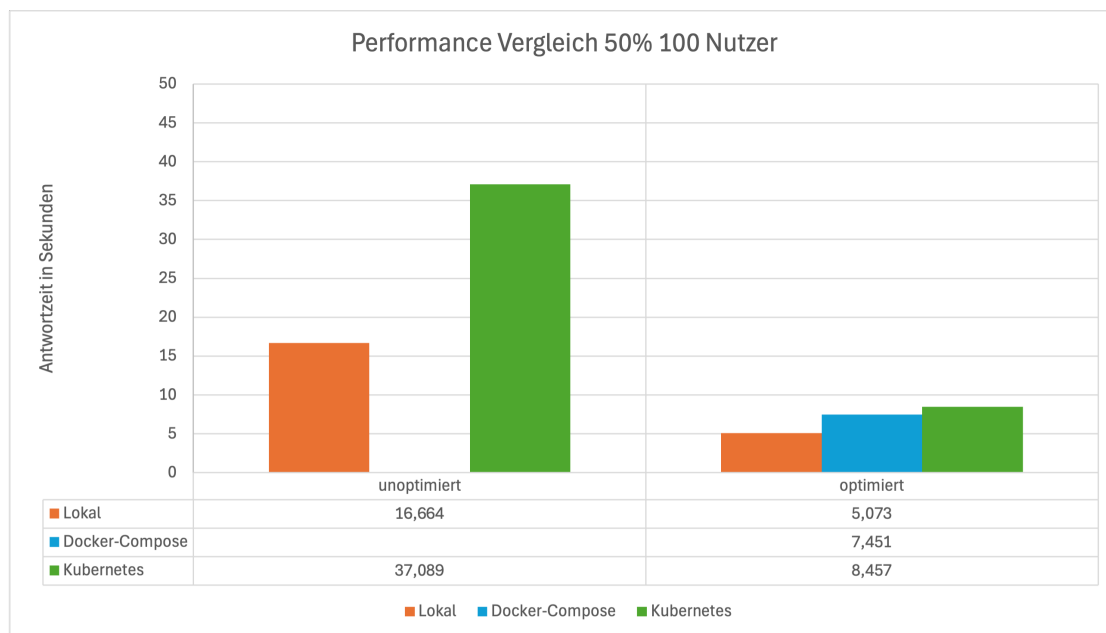


Abbildung 6.3: Performance Vergleich zwischen der lokalen, Docker-Compose und der Kubernetes Umgebung mit jeweils 100 gleichzeitigen Nutzern, im Vergleich die 50. Perzentil der Antwortzeit

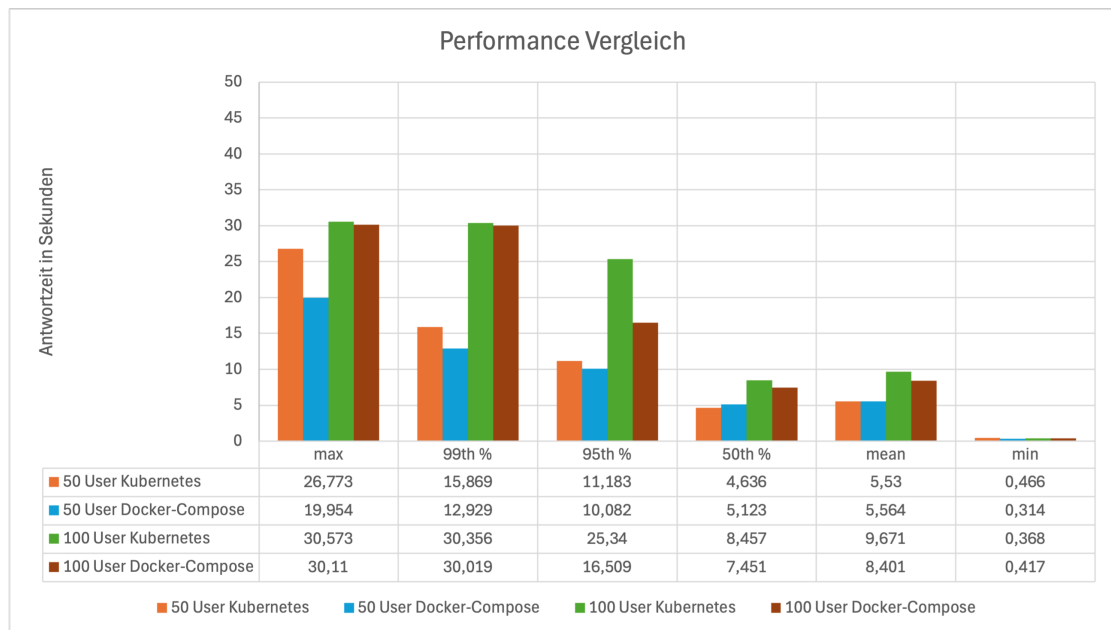


Abbildung 6.4: Performance Vergleich zwischen der Docker-Compose und der Kubernetes Umgebung mit jeweils 50 und 100 gleichzeitigen Nutzern

6.6. Bewertung der Ergebnisse

Die Ergebnisse zeigen klar, dass die Optimierung erfolgreich, aber auch notwendig war. Dennoch sind die Ergebnisse unter extrem hoher Auslastung unzureichend. In einer Klausur sollte ein Student nicht bis zu über 30 Sekunden auf eine Antwort warten müssen. Obwohl Kubernetes im schlechtesten Fall 53% schlechter ausfällt als Docker-Compose, sollte das System dennoch mit Kubernetes weiter entwickelt werden, da mit Kubernetes das System verteilt arbeiten kann und so eine wesentlich bessere Robustheit erhält. Dazu kommt, dass mit besserer Hardware oder mehr Ressourcen die Antwortzeiten ebenfalls stark fallen können, wie es der Vergleich mit der lokalen Umgebung aufzeigt. Des Weiteren zeigt die Evaluation auf, dass versteckte Tests, wie in den Anforderungen, erfolgreich eingebaut wurden und regulär nicht an das Frontend ausgehändigt werden. Die Fehlerfälle werden ebenfalls alle abgedeckt und sorgen für die gewünschte verbesserte Erläuterung des Fehlers für den Nutzer.

6.7. Grenzen der Evaluation

Die Performance-Tests wurden mit einem einzigen Testfall durchgeführt und decken keinen vollständigen Betrieb ab. Die gemessenen Werte sind daher abhängig von der Systemkonfiguration, der Hardware und dem genutzten Testfall. Zudem bildet die Evaluation keine vollständige Sicherheitsanalyse gegen alle denkbaren Angriffsszenarien ab.

7

Fazit

7.1. Zusammenfassung

Ziel dieser Bachelorarbeit war die Weiterentwicklung der bestehenden Check-API und die dafür notwendige Anpassung der umliegenden Systeme. Besonders relevant dabei war die Infrastruktur zur Unterstützung von versteckten Tests, sodass diese sicher ausgeführt werden können, ohne dabei von dem Nutzer einsehbar zu sein. Des Weiteren waren eine robustere Ausführung von Nutzercode, eine bessere Fehlerbehandlung und Fehlermeldung an den Nutzer und eine Verbesserung der Leistungsfähigkeit von großer Wichtigkeit.

Dafür wurde als Erstes die Verarbeitung der Aufgabenstellung erweitert, sodass versteckte Tests korrekt eingelesen und verarbeitet werden können. So kann der Parser nun sowohl öffentliche als auch versteckte Tests erkennen und getrennt voneinander in der Datenbank ablegen. So können im Programmablauf die Tests unterschiedlich gehandhabt werden, sodass versteckte Tests regulär nie an das Frontend geschickt werden. Ebenfalls wurden die Haupt-API und die Check-API so angepasst, dass diese versteckte Tests übermitteln und entgegennehmen können.

Die Check-API wurde des Weiteren so erweitert, dass nun versteckte Tests zusammen mit öffentlichen Tests und Nutzercode zu einer prüfbaren Einheit vereint werden. Zusätzlich verfügt die Check-API nun über eine erweiterte Fehlerbehandlung, welche unterschiedliche Szenarien unabhängig voneinander verarbeiten kann, um so dem Nutzer eine strukturierte, korrekte, sichere und aussagekräftige Fehlermeldung zu übermitteln.

7.2. Bewertung der Ergebnisse

Wie die Evaluation in Kapitel 6 zeigt, wurden die zentralen Anforderungen der Arbeit erfüllt. Versteckte Tests werden erfolgreich verarbeitet, ohne dabei die Inhalte der versteckten Tests an das Frontend zu schicken. Fehlerfälle wie Kompilierungsfehler, fehlgeschlagene öffentliche Tests, fehlgeschlagene versteckte Tests, Zeitüberschreitungen, Sicherheitsfehler und Laufzeitfehler werden unterschieden und in einem einheitlichen Format zurückgegeben.

Ebenso konnte die Robustheit erhöht werden. Zum einen wird der eingereichte Code vor der Ausführung auf kritische Befehle untersucht und zum anderen wird durch die Auslagerung der Ausführung in einen Unterprozess der Hauptprozess stärker vor bösartiger Codeausführung geschützt.

Die Performance-Evaluation in Abschnitt 6.5 zeigt ebenfalls eine Verbesserung gegenüber dem Systemzustand vor Beginn dieser Bachelorarbeit. Gleichzeitig wird deutlich, dass die Kompilierung und Ausführung trotz Optimierung rechenintensiv bleibt und besonders unter starker paralleler Last für hohe Antwortzeiten sorgt. Für die Nutzung während einer Klausur sollte das System vorab unter realistischen Lastbedingungen getestet und abhängig von den Ergebnissen skaliert werden.

7.3. Ausblick

Auf dieser Arbeit können weitere Verbesserungen umgesetzt werden. Zum einen kann zusätzlich zur Codeausführung ebenfalls die Kompilierung auf Unterprozesse ausgelagert werden, da auch die Kompilierung extrem rechenintensiv ist und so einen eigenen Kern nutzen kann, ohne von der restlichen Verarbeitung der Check-API gedrosselt zu werden. Des Weiteren können die Performance Evaluationen erweitert werden durch unterschiedliche Aufgaben, variierende Nutzerzahlen und klare Informationen über die verwendete Hardware. Besonders aussagekräftig wäre ein möglichst realistischer Klausurversuch, mit dem gemessen werden kann, wie oft der Service wirklich angefragt wird und wie die Check-API in solch einem kritischen Szenario tatsächlich performt.

Zusätzlich sollte die Sicherheitsprüfung weiter verbessert werden. Zwar werden gewisse verbotene Schlüsselwörter gefiltert, dennoch kann sowohl diese Liste erweitert werden, als auch die Erweiterung durch weitere Sicherheitskonzepte. Darunter fallen eine stärkere Isolation, Begrenzung von Ressourcen wie CPU, Speicher und Datenzugriff. Zusätzlich wäre eine Untersuchung eines Sicherheitsspezialisten hilfreich, um weitere Fehler und Schwächen in dem System aufzudecken, um auch diese beheben

zu können und das System somit nochmals sicherer machen zu können.

Auch die Rückmeldungen an Nutzer können weiter ausgebaut werden. Durch Nutzerfeedback kann festgestellt werden, wie eine gewisse Antwort noch präziser formuliert werden kann, um dem Nutzer genauer das Problem aufzuzeigen, ohne dabei dem Nutzer die korrekte Antwort offenzulegen.

Insgesamt bildet die Arbeit ein gutes Fundament dafür, CodeTask-Aufgaben sicherer, aussagekräftiger und unter den gemessenen Bedingungen effizienter zu prüfen. Die Check-API wurde von einem einfachen Prüfdienst zu einem strukturierten und robusteren Bestandteil der Lernplattform CodeTask weiterentwickelt.

Literatur

- [Ses17] Peter Sestoft. *Programming Language Concepts*. Springer, 2017. ISBN: 978-3-319-60788-7.
- [Was+18] Szymon Wasik u. a. “A Survey on Online Judge Systems and Their Applications”. In: *ACM Computing Surveys* 51.1 (2018). DOI: [10.1145/3143560](https://doi.org/10.1145/3143560).
- [EPF25] EPFL. *The Scala 3 compiler, also known as Dotty*. Version 3.6.3. 2025. URL: <https://github.com/scala/scala3> (besucht am 10. 06. 2026).
- [Doc26] Docker Inc. *Docker*. 2026. URL: <https://www.docker.com> (besucht am 16. 06. 2026).
- [Gat26] Gatling Corp. *Gatling*. 2026. URL: <https://gatling.io> (besucht am 27. 06. 2026).
- [Gök26] Konstantin Leon Göke. *Von Docker Compose zu Kubernetes: Evaluation der Zuverlässigkeit*. Bachelorarbeit, HTWG Konstanz. Zum Zeitpunkt der Abgabe unveröffentlichte Bachelorarbeit. 2026. URL: <https://github.com/...> (besucht am 19. 06. 2026).
- [The26a] The Kubernetes Authors. *Kubernetes*. 2026. URL: <https://kubernetes.io/de/> (besucht am 16. 06. 2026).
- [The26b] The Kubernetes Authors. *Kubernetes Nodes*. 2026. URL: <https://kubernetes.io/docs/concepts/architecture> (besucht am 19. 06. 2026).
- [The26c] The Kubernetes Authors. *Kubernetes Overview*. 2026. URL: <https://kubernetes.io/docs/concepts/overview> (besucht am 19. 06. 2026).
- [Hac] HackerRank. *HackerRank*. URL: <https://www.hackerrank.com> (besucht am 14. 06. 2026).
- [Lee] LeetCode. *LeetCode*. URL: <https://leetcode.com> (besucht am 14. 06. 2026).
- [Spoj] Spoj. *Spoj.com*. URL: <https://www.spoj.com> (besucht am 14. 06. 2026).